Мониторинг и поддержка в продакшене

Арзамасова Анна
Эксперт курса «Интеграция ИИ»



Проверка связи

Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти

Поставьте в чат:

\+ если меня видно и слышно

— если нет

Рекомендации

→ Если смотрите с компьютера

- Используйте браузеры Google Chrome или Microsoft Edge
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**

→ Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение на телефоне
- Предварительно проверьте, подходит ли ваше устройство для подключения к вебинару, по ссылке

Анна Арзамасова

О спикере:

- инженер в сфере авиации, сотрудник НИИ
- специалист по интеллектуальным системам управления транспортом
- преподаватель Московского Политеха
- Преподаватель и разработчик образовательных программ на стыке инженерии, программирования и искусственного интеллекта



Разбор ДЗ вебинара 11 · Завершаем модуль 3

ОРГМОМЕНТ

Возвращаемся к вебинару 1

Одна из трёх причин провала ИИ-проектов — отсутствие мониторинга: качество модели падает месяцами, а никто не замечает. Сегодня закрываем эту причину.



Цели занятия

После занятия вы сможете:

- **Объяснять,** чем различаются инфраструктурный, продуктовый и модельный мониторинг;
- **Выбирать** метрики для каждого уровня и определять пороги по историческим данным;
- **Добавлять** в FastAPI-сервис middleware со структурированным логированием и счётчиком запросов;
- **Проектировать** эскиз дашборда и сценарии алертов: метрика, порог, серьёзность, действие.

План вебинара

1. Типы мониторинга
2. Метрики
3. Дашборды
4. Алертинг и логирование
5. Практика: метрики в сервисе
6. Квиз по модулю 3

Типы мониторинга

ЧАСТЬ 1

Сервис может быть технически исправен и при этом бесполезен — модель внутри деградировала



Инфраструктурный мониторинг

Жив ли сервис и хватает ли ему ресурсов: загрузка CPU, память, доступность, диск.



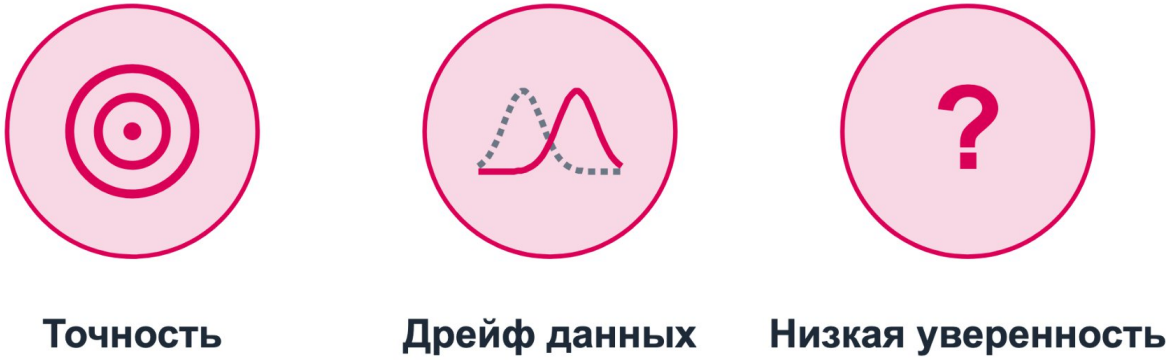
Продуктовый мониторинг

Пользуются ли сервисом и достигает ли он бизнес-цели: число запросов, конверсия, вовлечённость.



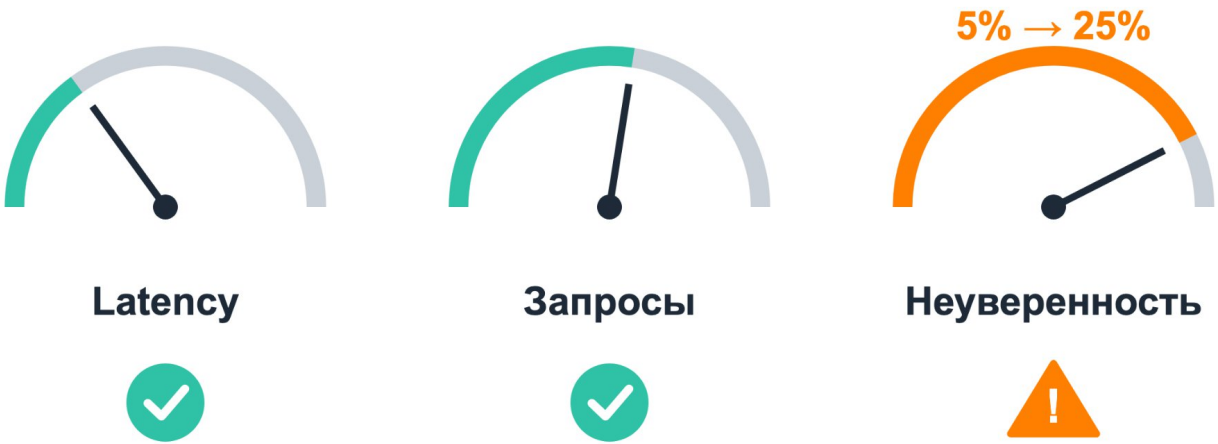
Модельный мониторинг

Продолжает ли модель делать хорошие предсказания: точность на реальных данных, дрейф входных данных, доля низкоуверенных предсказаний.

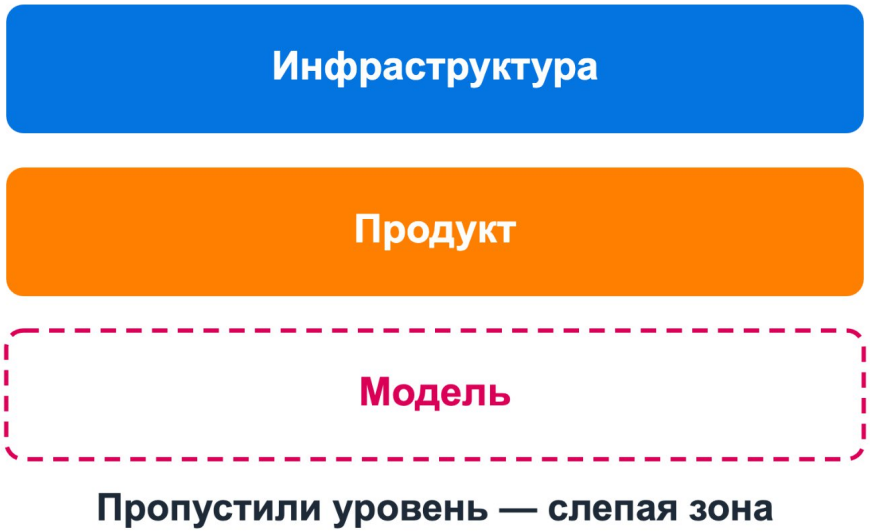


Пример: сервис антифрода

Latency 120 мс — отлично. Число запросов стабильно — нормально. Доля низкоуверенных предсказаний выросла с 5% до 25% за месяц — тревожный сигнал.



Три уровня мониторинга независимы — упущение любого создаёт слепую зону

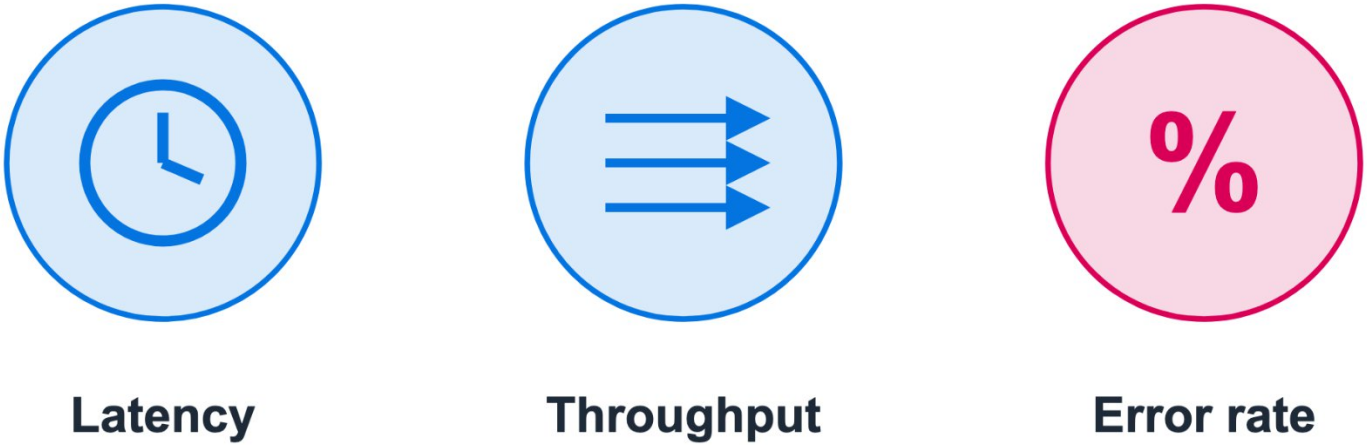


Метрики

ЧАСТЬ 2

Инфраструктурные метрики

Latency — задержка ответа. Throughput — пропускная способность. Error rate — доля запросов с ошибкой.

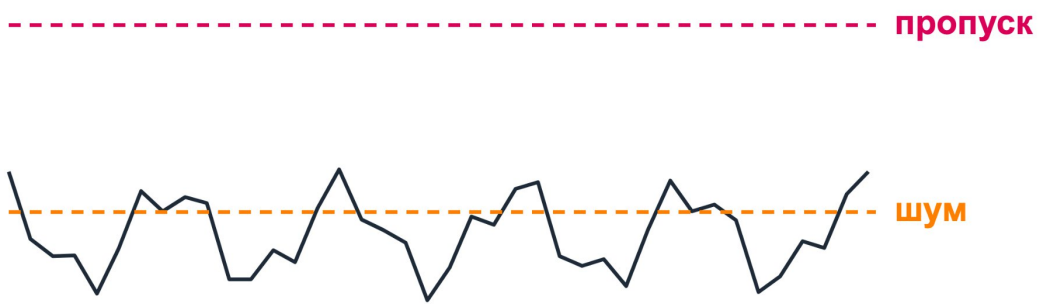


Модельные метрики

Accuracy и метрики качества из вебинара 7 — но часто нет истинной метки сразу. Косвенные сигналы: data drift, доля низкоуверенных предсказаний.



Метрика бесполезна без порога. Слишком узкий порог — шум, слишком широкий — пропущенные проблемы



Как калибровать пороги

Опирайтесь на исторические данные: типичное значение метрики за последние недели в спокойный период + запас.

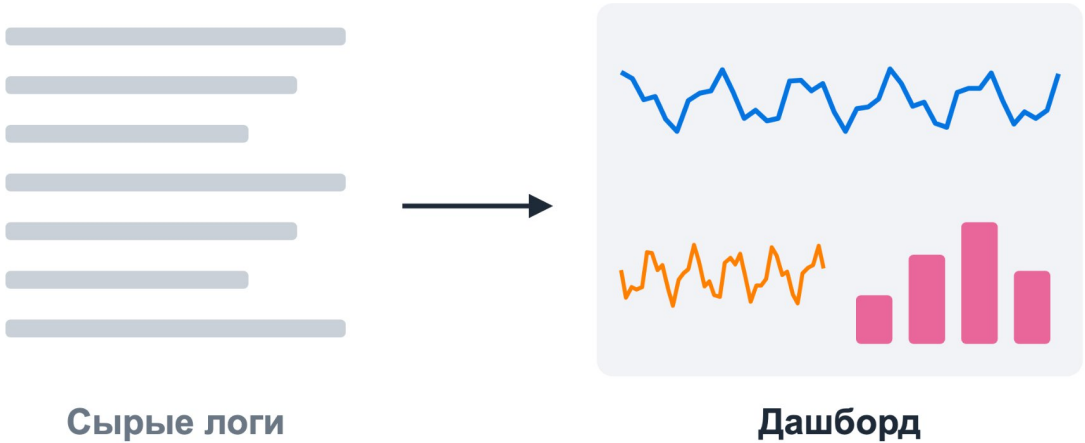


Дашборды

ЧАСТЬ 3

Определение

Визуальная панель с метриками системы в виде графиков и показателей — для быстрого понимания состояния без чтения сырых логов.

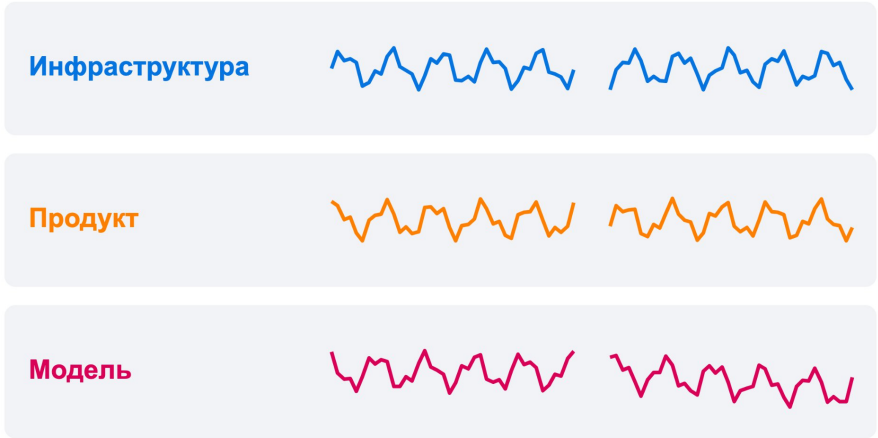


Три блока дашборда

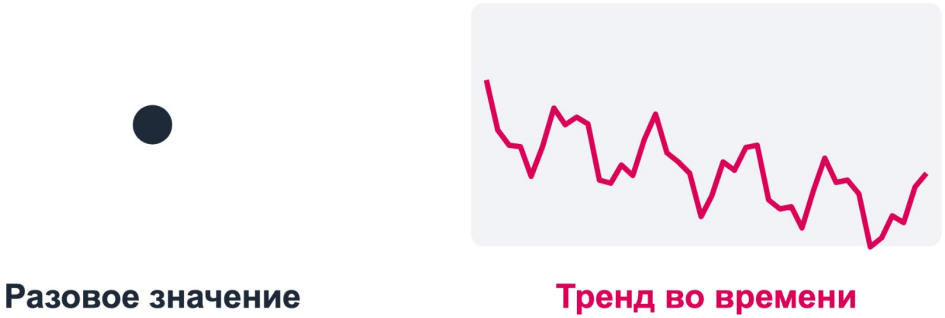
Инфраструктура
latency, throughput, error rate

Продукт
число запросов, бизнес-метрика

Модель
уверенность предсказаний, признаки дрейфа



Деградация происходит постепенно — важен тренд во времени, а не разовое значение



Разовое значение

Тренд во времени

Prometheus и Grafana (обзор)

Prometheus периодически собирает числовые метрики. Grafana строит дашборды поверх этих данных. В курсе — не разворачиваем полный стек, только структура и простой сбор метрик в коде.



сбор метрик → графики

Алертинг и логирование

ЧАСТЬ 4

Алертинг

Автоматическое оповещение, когда метрика выходит за порог, без ручного наблюдения за дашбордом.



Хороший алерт отвечает на три вопроса

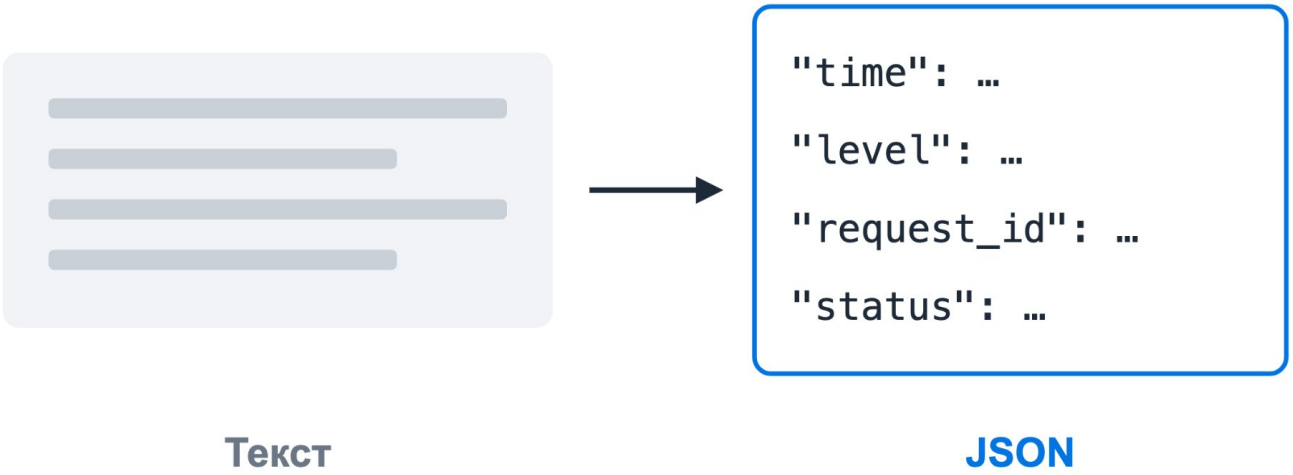
Что именно произошло

Насколько это серьёзно

Что делать дальше

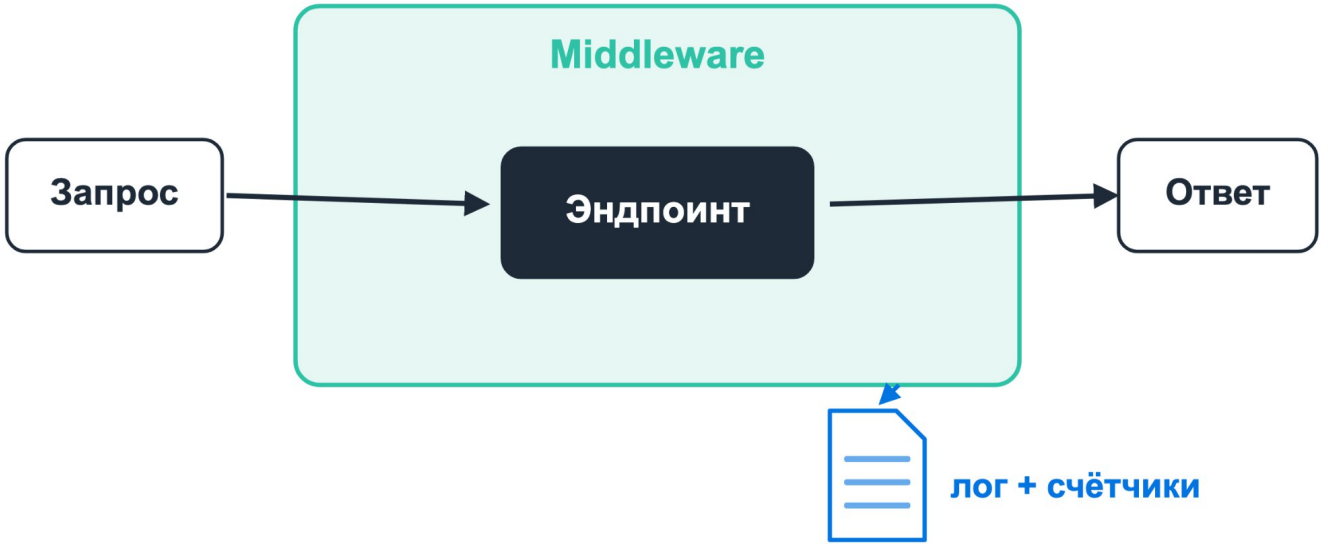
Structured logging

Запись логов в машиночитаемом формате (JSON) с явными полями — время, уровень важности, идентификатор запроса.



Middleware

Код, выполняющийся для каждого запроса до/после основной логики — удобное место для логирования и сбора метрик без дублирования.



Метрики и дашборд для сервиса

ПРАКТИКА

Шаг 1



Лайв-кодинг: добавляем middleware в FastAPI-сервис, логирующий время ответа, статус-код и вход каждого запроса.

```python
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pickle

X, y = make_classification(n_samples=1000, n_features=6, n_informative=4, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

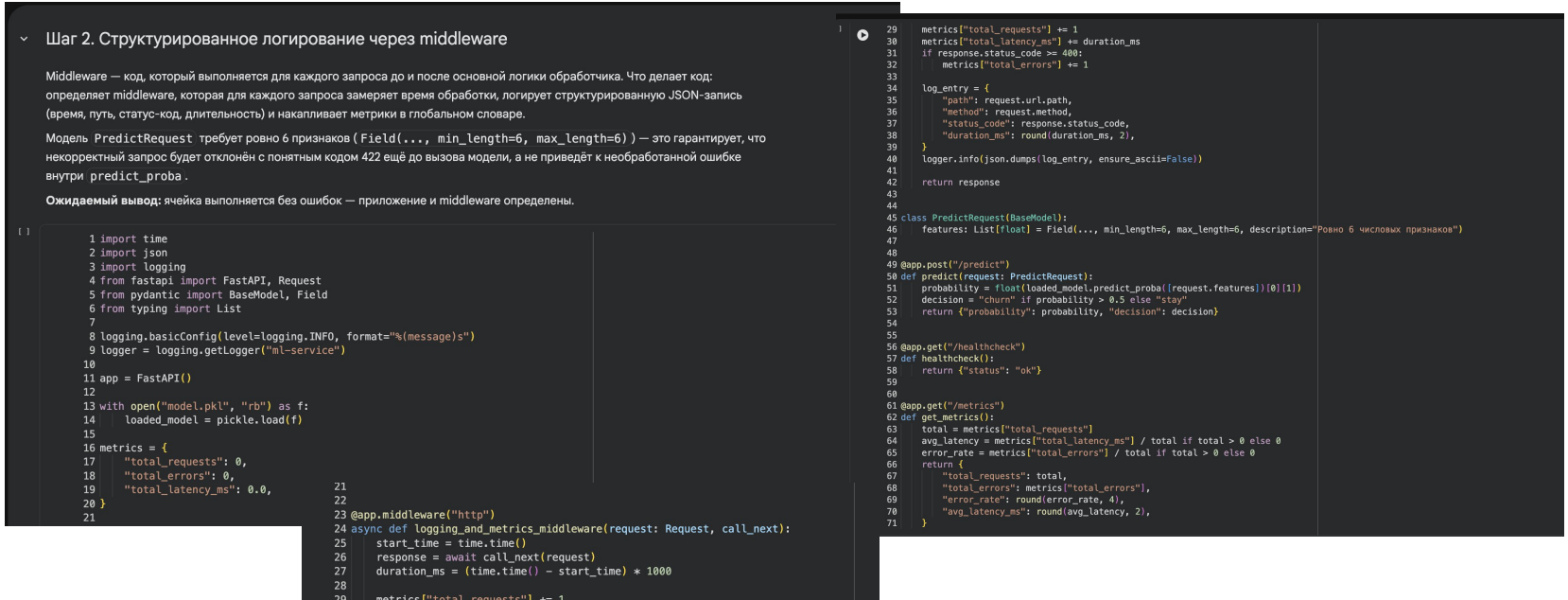
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
acc = accuracy_score(y_test, model.predict(X_test))
print("Accuracy:", round(acc, 3))

with open("model.pkl", "wb") as f:
    pickle.dump(model, f)
```

```
Accuracy: 0.925
```

Шаг 2

Лайв-кодинг: добавляем простой счётчик запросов в памяти сервиса — без внешней инфраструктуры.



Шаг 2. Структурированное логирование через middleware

Middleware — код, который выполняется для каждого запроса до и после основной логики обработчика. Что делает код: определяет middleware, которая для каждого запроса замеряет время обработки, логирует структурированную JSON-запись (время, путь, статус-код, длительность) и накапливает метрики в глобальном словаре.

Модель `PredictRequest` требует ровно 6 признаков (`Field(..., min_length=6, max_length=6)`) — это гарантирует, что некорректный запрос будет отклонён с понятным кодом 422 ещё до вызова модели, а не приведёт к необработанной ошибке внутри `predict_proba` .

Ожидаемый вывод: ячейка выполняется без ошибок — приложение и middleware определены.

```python
import time
import json
import logging
from fastapi import FastAPI, Request
from pydantic import BaseModel, Field
from typing import List

logging.basicConfig(level=logging.INFO, format="%(message)s")
logger = logging.getLogger("ml-service")

app = FastAPI()

with open("model.pkl", "rb") as f:
    loaded_model = pickle.load(f)

metrics = {
    "total_requests": 0,
    "total_errors": 0,
    "total_latency_ms": 0.0,
}


@app.middleware("http")
async def logging_and_metrics_middleware(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request)
    duration_ms = (time.time() - start_time) * 1000

    metrics["total_requests"] += 1
    metrics["total_latency_ms"] += duration_ms
    if response.status_code >= 400:
        metrics["total_errors"] += 1

    log_entry = {
        "path": request.url.path,
        "method": request.method,
        "status_code": response.status_code,
        "duration_ms": round(duration_ms, 2),
    }
    logger.info(json.dumps(log_entry, ensure_ascii=False))

    return response


class PredictRequest(BaseModel):
    features: List[float] = Field(..., min_length=6, max_length=6, description="Ровно 6 числовых признаков")

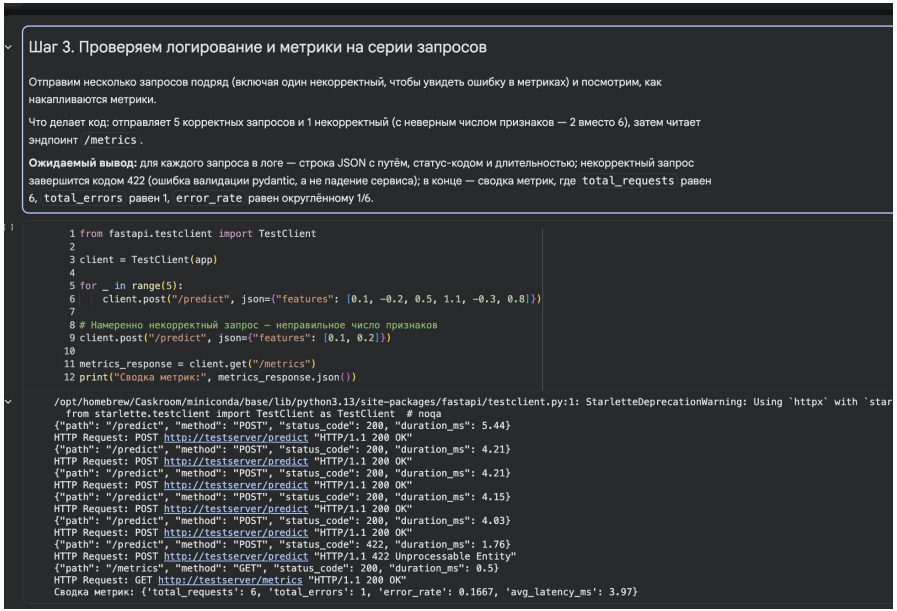

@app.post("/predict")
def predict(request: PredictRequest):
    probability = float(loaded_model.predict_proba([request.features])[0][1])
    decision = "churn" if probability > 0.5 else "stay"
    return {"probability": probability, "decision": decision}


@app.get("/healthcheck")
def healthcheck():
    return {"status": "ok"}


@app.get("/metrics")
def get_metrics():
    total = metrics["total_requests"]
    avg_latency = metrics["total_latency_ms"] / total if total > 0 else 0
    error_rate = metrics["total_errors"] / total if total > 0 else 0
    return {
        "total_requests": total,
        "total_errors": metrics["total_errors"],
        "error_rate": round(error_rate, 4),
        "avg_latency_ms": round(avg_latency, 2),
    }
```

Шаг 3

Групповое проектирование: эскиз дашборда — какие метрики, в каком блоке, какой тип графика.



Шаг 3. Проверяем логирование и метрики на серии запросов

Отправим несколько запросов подряд (включая один некорректный, чтобы увидеть ошибку в метриках) и посмотрим, как накапливаются метрики.

Что делает код: отправляет 5 корректных запросов и 1 некорректный (с неверным числом признаков — 2 вместо 6), затем читает эндпоинт `/metrics`.

Ожидаемый вывод: для каждого запроса в логе — строка JSON с путём, статус-кодом и длительностью; некорректный запрос завершится кодом 422 (ошибка валидации pydantic, а не падение сервиса); в конце — сводка метрик, где `total_requests` равен 6, `total_errors` равен 1, `error_rate` равен округлённому 1/6.

```
from fastapi.testclient import TestClient

client = TestClient(app)

for _ in range(5):
    client.post("/predict", json={"features": [0.1, -0.2, 0.5, 1.1, -0.3, 0.8]})

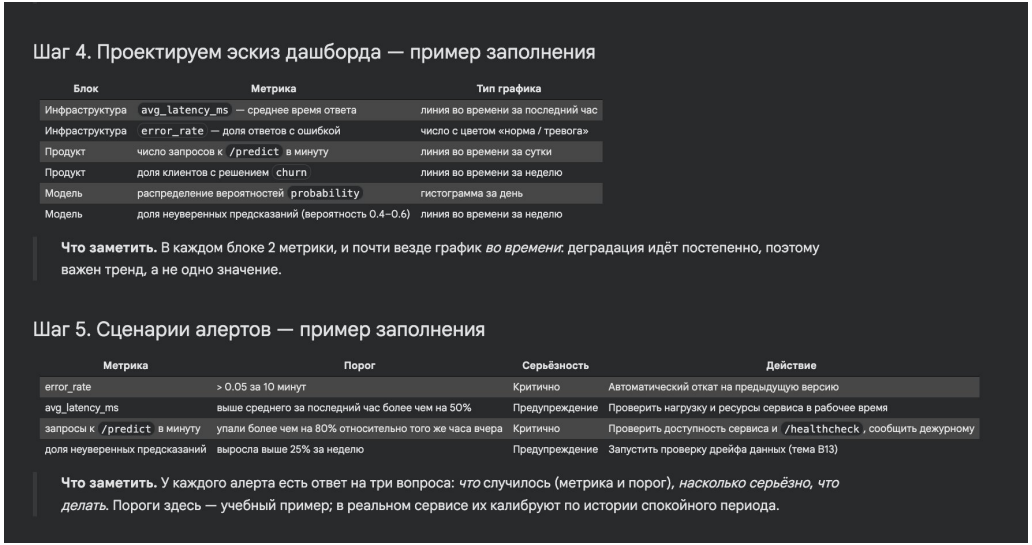
# Намеренно некорректный запрос — неправильное число признаков
client.post("/predict", json={"features": [0.1, 0.2]})

metrics_response = client.get("/metrics")
print("Сводка метрик:", metrics_response.json())
```

```
/opt/homebrew/Caskroom/miniconda/base/lib/python3.13/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `star
  from starlette.testclient import TestClient as TestClient  # noqa
{"path": "/predict", "method": "POST", "status_code": 200, "duration_ms": 5.44}
HTTP Request: POST http://testserver/predict "HTTP/1.1 200 OK"
{"path": "/predict", "method": "POST", "status_code": 200, "duration_ms": 4.21}
HTTP Request: POST http://testserver/predict "HTTP/1.1 200 OK"
{"path": "/predict", "method": "POST", "status_code": 200, "duration_ms": 4.21}
HTTP Request: POST http://testserver/predict "HTTP/1.1 200 OK"
{"path": "/predict", "method": "POST", "status_code": 200, "duration_ms": 4.15}
HTTP Request: POST http://testserver/predict "HTTP/1.1 200 OK"
{"path": "/predict", "method": "POST", "status_code": 200, "duration_ms": 4.03}
HTTP Request: POST http://testserver/predict "HTTP/1.1 200 OK"
{"path": "/predict", "method": "POST", "status_code": 422, "duration_ms": 1.76}
HTTP Request: POST http://testserver/predict "HTTP/1.1 422 Unprocessable Entity"
{"path": "/metrics", "method": "GET", "status_code": 200, "duration_ms": 0.5}
HTTP Request: GET http://testserver/metrics "HTTP/1.1 200 OK"
Сводка метрик: {'total_requests': 6, 'total_errors': 1, 'error_rate': 0.1667, 'avg_latency_ms': 3.97}
```

Шаг 4

Опишите 3–5 сценариев алертов: метрика, порог, серьёзность, действие при срабатывании.



Шаг 4. Проектируем эскиз дашборда — пример заполнения

Блок	Метрика	Тип графика
Инфраструктура	`avg_latency_ms` — среднее время ответа	линия во времени за последний час
Инфраструктура	`error_rate` — доля ответов с ошибкой	число с цветом «норма / тревога»
Продукт	число запросов к `/predict` в минуту	линия во времени за сутки
Продукт	доля клиентов с решением `churn`	линия во времени за неделю
Модель	распределение вероятностей `probability`	гистограмма за день
Модель	доля неуверенных предсказаний (вероятность 0.4–0.6)	линия во времени за неделю

> **Что заметить.** В каждом блоке 2 метрики, и почти везде график *во времени*: деградация идёт постепенно, поэтому важен тренд, а не одно значение.

Шаг 5. Сценарии алертов — пример заполнения

Метрика	Порог	Серьёзность	Действие
error_rate	> 0.05 за 10 минут	Критично	Автоматический откат на предыдущую версию
avg_latency_ms	выше среднего за последний час более чем на 50%	Предупреждение	Проверить нагрузку и ресурсы сервиса в рабочее время
запросы к `/predict` в минуту	упали более чем на 80% относительно того же часа вчера	Критично	Проверить доступность сервиса и `/healthcheck`, сообщить дежурному
доля неуверенных предсказаний	выросла выше 25% за неделю	Предупреждение	Запустить проверку дрейфа данных (тема B13)

> **Что заметить.** У каждого алерта есть ответ на три вопроса: *что* случилось (метрика и порог), *насколько серьёзно*, *что делать*. Пороги здесь — учебный пример; в реальном сервисе их калибруют по истории спокойного периода.

Если не работает

Middleware не срабатывает на каждый запрос
проверьте @app.middleware("http") и await call_next(request)

Непонятно, что включать в дашборд
три блока — инфраструктура/продукт/модель, по 2–3 метрики

Сложно сформулировать порог
используйте относительный — отклонение от среднего за час/день более X%

Вы узнали

Главная мысль:

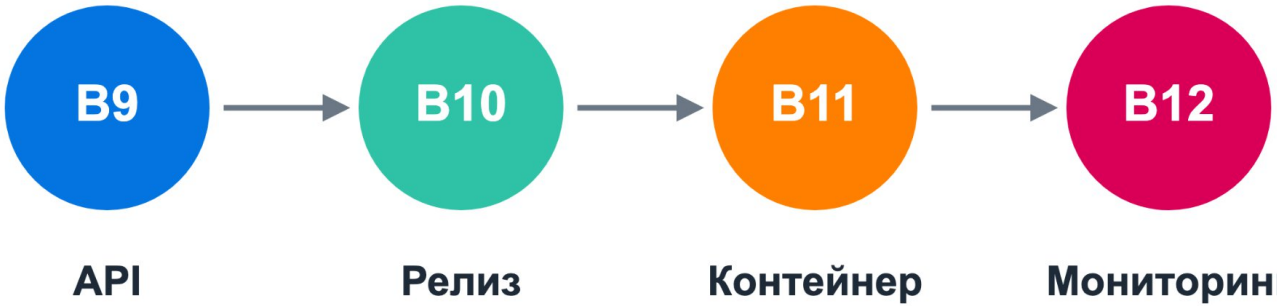
Сервис может отвечать быстро и без ошибок, а модель внутри — деградировать, поэтому наблюдать нужно три уровня сразу: инфраструктуру, продукт и модель, а каждой метрике нужен порог и понятный алерт.

На практике: лучше пять хорошо выбранных метрик с разумными порогами, чем пятьдесят метрик, за которыми никто не следит.

От API до наблюдаемой системы в проде

ИТОГИ МОДУЛЯ 3

API (B9) → безопасный релиз (B10) → переносимое окружение (B11) → наблюдаемость и алертинг (B12)



Тест по модулю 3

10 вопросов, 10 минут. Закрепление материала четырёх вебинаров модуля.

Домашнее задание

Добавьте логирование и метрики в ваш сервис, спроектируйте эскиз дашборда с текстовым описанием.

Мост к модулю 4

На вебинаре 13 — подробнее про дрейф данных и концепта, стратегии переобучения: что делать, когда мониторинг сигнализирует о проблеме.

СПАСИБО

До встречи на вебинаре 13

Мониторинг и поддержка в продакшене

Арзамасова Анна
Эксперт курса «Интеграция ИИ»

